CSC-696D, Takanori Fujiwara
Final project report.
Novel visualization techniques for git data.
Source Code: https://github.com/rowan907/CSC-696D-Final-Project.
Hosted project: https://rowan907.github.io/CSC-696D-Final-Project.

0.1 Problem Description

In this project we created several novel visual analytic systems for git data. The visualizations we propose will help developers quickly gain an understanding of git repositories. In particular, we seek to give users an understanding of how a repo changes over time. We will focus on a few key questions.

1. Who are the main contributors to a repository?
2. What were the development priorities for a repository at any given time?
3. Can we identify a time when AI-assisted development became popular on a given repository?
4. What is the best visualization for giving developers an understanding of the evolution of project structure over time?

0.2 Motivation

Developers often work with large amounts of code and are forced to quickly learn the customs and conventions of many repositories, this process can be very time-consuming. By giving developers tools for quickly visualizing the history of a repository, we can immediately give them an understanding of conventions, key contributors, and etiquette.

Many visualizations have been developed over the years to work with git data, in this project we developed a handful of new visualizations by first studying visualizations that others have developed, then either refining them to improve their visual clarity or augmenting them with additional data.

We developed 3 visualizations for this project.

1. A commit graph showing branches of the repository. Users may filter through selecting portions of commits for the graphs below to use. In a workflow, this focuses on looking at a portion of commits to see differences in lines of code, authorship, files, and commit messages. Users may also filter by authors. This project has an emphasis on authorship as described in Git-Truck's user-evaluated metrics.

2. A 'stabilized' Sediment Graph based on the work of Erik Bernhardsson [4]. Bernhardsson proposes a unique way to understand the age of the contents of a repo, improved his designs by changing the stacking behavior so as rows are added we allow the graph to grow in the positive and negative y directions. This improves the stability of the graph by keeping individual lines of sediment more horizontal. Reducing the visual interference caused by the sediment layers traveling diagonally up the graph, making there width harder to determine.

   We also (based on feedback we received from our peers), added to new views to this visualization. In addition to dividing the layers of sediment by code age we also provided the option to divide the sediment by author, supporting both total lines of code by author and number of commits by author. These views may be an even better use case then code age, as authorship is generally a more important metric.

   In addition, we added interactions to this graph. First, hovering on each sediment gives users information on the time range, authorship, or number of commits. Rather than having users estimate the values for each of these, we provide them on hover for visual clarity. We also allow users to extract sediments from the graph, particularly remove outliers. For example, if one author has a large number of lines of code written, the author is extracted to normalize the distribution of all other authors. Users may use this in order to increase the expressivity of visual area and also to find the top contributors of the repository.

3. Word Cloud-style view that allows users to select many commits via a time window selection and see a word cloud of the commit messages. This will allow users to quickly see the development focuses of a repo at a particular time window. This is inspired by a view from [1]. We seek to enhance their view by adding a time window selection allowing these clouds to be generated for any section of the commit history. We also provide users the ability to click a word from the word cloud and highlight the commits that include that word. Allowing users to quickly see what changes dealt with certain topics.

   We augment the Word Cloud through a simple tf-idf weighting of terms. Rather than having users sift through all terms manually, we attempt to give users important, unique terms to sift through. At times, authors and referenced libraries are identified, providing a higher level of understanding than the most frequently used terms.

4. File Cluster Map. A grouping-based visualization based on a force directed graph that clusters files or directories that are often modified together, this helps users understand which parts of the codebase are most tightly coupled. We animated this view to show

the evolution of the repo over time and coupled it with a timeline view allowing users to scrub to particular sections of the animation.

Nodes here correspond to both files and directories. Directories are opaque nodes, and files are transparent nodes with dotted outlines. Directories can be expanded into files, and files can be collapsed into files. The directory view gives users the ability focus on higher-level abstract details. For instance, is documentation being created alongside tests? With time animation, this becomes apparent for users.

With all node-link diagrams, the number of links dramatically impacts the visual expressivity of the graph. This project's file cluster map is no different. Therefore, we allow users to change the number of files shown when a folder is expanded. The files are selected by the number of lines of code they contain so as to select the most important documents.

0.3 Related Research Papers

- [1] — This work shows an all-in-one visual analytic tool for examining git history and provides a lot of great background on git. Many techniques here may be useful; in particular, the paper provides a lot of detail on the technical aspects of creating visualizations with git metadata. We also took inspiration from this paper for our second proposed visualization as discussed above.

- [2] — Git-Truck is a hierarchical file-organization-oriented visualization tool that represents git repositories as different treemaps. Each treemap has its nodes sized by the number of lines of code in the repository, and the color of its nodes can represent a variety of interesting factors such as number of commits, top contributors, or even the last change day. This is showing the current structure of a repository, rather than the temporal focus we have in this project. Mainly, we emphasize the importance of seeing the historical development of features in a repository. Git-Truck also provides a set of metrics that provide guidance for this paper's implementation: number of commits, single author, and top contributor. Here, this project highlights the importance of authorship for repository exploration.

- [3] — This paper gives another approach to visualizing code changes that relies on semantic analysis. Though this paper's focus is more on visualizing individual commits and discrete changes rather than the type of broad analysis of larger repositories we did, it gives some good background on existing visual analytic techniques.

0.4 Technical Approach

Our technical approach is at a high level

1. Collect `.git` files containing git meta-data from repos we wish to study.
2. Use a pre-processing script to execute git commands to query this data and create a `.json` file for each repo containing all commits we want to analyze and any useful meta-data like branch, parent, files changed etc.
3. Load our `.json` files at run time when the users selects a particular repo they want to analyze.
4. Build our visualizations from the processed data.

Our pre-processing script lives inside the project (`app/scripts/preprocessgit.mjs`) and can be triggered manually with `npm run preprocess:git`. The script uses JavaScript's `exec` function to run git with the flag `--git-dir=[.git file]`, this command allows git to behave for most commands as if it's inside the project we're targeting despite only having access to the metadata. This saves us having to pull the whole project we wish to study.

We then run a `git log` to get all commit hashes across all branches (for performance reasons we cap this at the 4500 most recent commits when more are available). Finally, we run git commands on every individual commit to get information like predecessor and branch. The information we pull from each commit is bellow and is used to support all three visualizations.

```
type Commit {
  hash: string;
  parents: Commit[];
  children: Commit[];
  author: string;
  date: string;
  subject: string;
  branch: string | null;
  merge?: string | null;
  files?: CommitFile[];
}
```

```
type CommitFile {
  additions: number;
  deletions: number;
  path: string;
}
```

We then include all json files in the `public` directory of our frontend app we configure React to serve all files in this directory, so that at runtime we can query them as needed.

We do this because these files are fairly large and not bundling all of them with our app directly reduces load time. [1]

File Cluster Map

The File Cluster Map is built on a D3 force-directed graph. A graph-construction pass iterates over all commits: for each commit touching 50 or fewer files, every pair of co-modified files receives a weighted edge whose weight counts the number of times that pair appears together across all commits; edges with weight below 2 are discarded. Files beyond a configurable per-directory cap are collapsed into their parent directory node, keeping the graph readable by default while still allowing users to expand directories on demand. Node radius is scaled by total lines of code using a square-root scale (capped between 5 and 28 px), so the most-touched files are visually prominent without dominating the layout.

The force simulation uses four forces: a charge repulsion of -180 to spread nodes apart, a centering force, a link force whose strength is proportional to co-occurrence weight, and a collision force using each node's radius. Directory nodes are rendered as solid filled circles; file nodes use a dashed stroke outline to visually distinguish the two levels of the hierarchy. Link width is also scaled by co-occurrence weight, reinforcing the coupling signal through both color and thickness.

To animate the graph over time, the component is driven by a filtered commit array from the timeline brush: as the brush window advances, the graph is rebuilt from only the commits in that window, and the force simulation is restarted. A stable color map assigns each top-level directory a consistent hue across all animation frames so that clusters remain visually identifiable as the graph evolves.

Sediment (Stream) Graph

The Sediment Graph is implemented using D3's stack generator with three selectable data modes. In *era mode* the commit timeline is divided into 32 equal cohorts; each file is attributed to the cohort in which it was first introduced, and deletions erode the surviving line count of that birth cohort. In *author lines-of-code mode* each file is owned by the first among the top-50 authors (by total additions) to touch it, with deletions eroding that author's count; in *author commits mode* cumulative commit counts per author are plotted instead of lines of code. All three modes produce a matrix of 200 evenly spaced time samples by cohort, which is fed to the D3 stack generator.

[1] If you're curious you can see the raw json files by navigating to `https://rowan907.github.io/CSC-696D-Final-Project/flask_commits.json`

The key design choice distinguishing our graph from Bernhardsson's original is the stacking offset. We use `stackOffsetWiggle` with `stackOrderInsideOut`, which centers the stack around a horizontal baseline and orders bands to minimize visual slope. This keeps individual sediment bands close to horizontal regardless of their relative magnitude, making width (and therefore surviving line count) easier to read than in a bottom-anchored stack where upper bands travel diagonally across the chart.

Paths are rendered with `curveBasis` smoothing and colored by HSL values computed from cohort index. Hovering a band dims all others to 0.3 opacity and shows a tooltip with the cohort label, era or author metadata, and net line or commit count. Clicking a band extracts it from the main graph into a separate line panel below with its own y-axis; this is particularly useful for removing a dominant author whose band would otherwise compress all others into an unreadable sliver. A double-click on the line panel restores all extracted cohorts.

To speed up implementation we used a few AI tools, primarily Claude Haiku/Sonnet/Opus 4.x, as well as Qwen 3.6 running locally for smaller changes and refactors.

0.5 Technical Challenges

We initially tried to get all the information we'd need from the git log command however this involved manually parsing commit messages to try and infer things like merges which was not completely accurate. In the end we settled on the approach of getting the commit hashes from the log and running a git command for every commit to get the additional data we needed. This approach was much slower but more reliable, and since the processing script only needs to run once the slowdown was acceptable.

0.6 Selected Dataset(s)

We selected 4 medium size repos to examine in this project.

- Flask
- Click
- HTTPX
- Requests

0.7 Expected results and evaluation methodology

Expected results. The primary deliverable of this project is a deployed web application demonstrating the four visualizations described above against four real-world Python repositories (Flask, Click, Requests, HTTPX). Each visualization targets one or more of the research questions posed in Section 1; we state our concrete expectations below.

- *Stabilized Sediment Graph — author mode (RQ1).* The commit graph's author filter answers RQ1 at the level of individual commits, but the sediment graph in author mode answers it at the level of *surviving impact*: each band's width encodes how many lines written by a given author are still present in the codebase today. This distinguishes authors who made large early contributions that were later refactored away from those whose code forms the lasting foundation of the project—a distinction raw commit counts cannot make. The extract-sediment interaction reinforces this: pulling out a dominant author's band reveals the relative contributions of everyone else, turning a graph that might otherwise be unreadably skewed into one that communicates the full contributor hierarchy. We expect a small number of "core" contributors to dominate each repository, consistent with the power-law distributions reported in the Git-Truck literature.

- *Stabilized Sediment Graph — age-of-code mode (RQ3).* The age-of-code view should show that most surviving lines in mature repositories were written in early cohorts, with recent cohorts contributing smaller but growing fractions—a signature of stable, incrementally maintained projects. We additionally expect the sediment graph and word cloud to jointly surface a visible inflection point—an era in which commit subjects begin containing AI-assistance markers such as "co-authored-by" or tool names—providing evidence for or against RQ3.

- *Word Cloud with tf-idf weighting (RQ2).* Restricting the time window to a known development phase (e.g. a major version release) should produce a cloud whose dominant terms closely match the human-readable changelog for that release. The tf-idf weighting should suppress generic filler words and surface release-specific terminology or contributor handles that plain frequency counts would bury.

- *File Cluster Map (RQ4).* Tightly coupled subsystems (e.g. test files co-modified with their implementation counterparts) should form visually distinct clusters in the force-directed layout. Animating the graph over time should show the emergence of new modules and the decoupling or deprecation of old ones, giving developers an intuitive picture of architectural evolution.

Evaluation methodology. We evaluate our visualizations through an informal expert survey. Participants are professional software engineers recruited from the authors' networks; each was shared a link to the hosted application and asked four open-ended questions:

1. What git tools do you currently use to understand development history (e.g. IDE integrations, `git log`)?
2. What questions are most important to you before beginning work on an unfamiliar project?
3. Do you think a visual analytic system would be helpful for getting an overview of a git repository?
4. Do any of the visualizations from the project help you answer the questions from (2)? Why or why not?

Questions 1 and 2 establish a baseline of the tools and information needs participants already have, providing context for interpreting their reactions to our system. Question 3 gauges overall receptiveness to visual analytics for git data, and question 4 directly probes whether our specific visualizations address real developer needs. Responses are analyzed qualitatively: we look for recurring themes across participants and note which visualizations are cited as useful or insufficient relative to the needs surfaced in question 2.

0.8 Evaluation results.

We collected responses from two professional software engineers. Participant A primarily uses Azure DevOps / TFS rather than raw git tooling, and their most-used feature is a file-history view showing the change history of individual files. Participant B is a command-line-oriented git user whose go-to command is `git log --oneline --graph --all --decorate`, and does not use IDE integrations or other visual tools.

On question 2, both participants converged on the same underlying concern despite different framings: they want to understand *what code is coupled* and *how it has changed*, before touching any of it. Participant A asked specifically when and how the area they would be working on changed, and which other areas might be affected by their changes. Participant B framed this in terms of inferring functional and non-functional requirements, including thinking through the implications for future developers and maintainers.

Both participants were receptive to visual analytics (question 3). Participant A noted that it provides a quick understanding of what is happening in a repo rather than "staring at a wall of text." Participant B said it would be "amazing."

On question 4, the *File Cluster Map* was cited by both participants as the most directly useful visualization relative to their stated needs. Participant A called it the closest to answering question 2 and said it "would give a visual understanding of what areas of the code to pay attention to when starting to work." Participant B highlighted the *Commit Graph* as feeling like the most important visualization overall, while also noting that the File Cluster Map was valuable; they observed that the commit graph does not directly answer question 2 but is nonetheless highly useful in practice.

The two responses are consistent with each other and with our expectations: the File Cluster Map addresses the most practically urgent developer question (coupling and impact analysis), while the Commit Graph serves as a useful structural overview.

References

[1] Youngtaek Kim, Jaeyoung Kim, Hyeon Jeon, Young-Ho Kim, Hyunjoo Song, Bohyoung Kim, and Jinwook Seo, "Githru: Visual Analytics for Understanding Software Development History Through Git Metadata Analysis," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 656–666, 2021. doi:10.1109/TVCG.2020.3030414

[2] K. Højelse, T. Kilbak, J. Røssum, E. Jäpelt, L. Merino, and M. Lungu, "Git-Truck: Hierarchy-Oriented Visualization of Git Repository Evolution," in *Proc. 30th IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2022. [PDF]

[3] Lei Chen, Michele Lanza, and Shinpei Hayashi, "ChangePrism: Visualizing the Essence of Code Changes," *arXiv preprint arXiv:2508.12649*, 2025. arXiv:2508.12649

[4] E. Bernhardsson, "Git of Theseus: Analyzing how a codebase changes over time," *GitHub repository*, 2015. https://github.com/erikbern/git-of-theseus